instituto superior de engenharia de lisboa

LMATE

Programação

Inverno 2026-2027

Trabalho 13

1

Exercício 6 - Muitos preços

- Considere os dez preços de produtos que se seguem:

```
97 70 15 38 92 30 86 68 6 80
```

- Todos os produtos estão com um desconto de 10%.
- Escreva um programa que calcule o preço a pagar por cada produto.

Solução

Programa (no ficheiro `exercicio_6_muitos_precos.py`)

Escreva e execute este programa. Confirme o seu *output*.

```
def preço_a_pagar(preço, desconto):

    return preço * (1.0 - desconto/100)

desconto = 10 # este valor é em percentagem

preço_a_pagar_1  = preço_a_pagar(97, desconto)
preço_a_pagar_2  = preço_a_pagar(70, desconto)
preço_a_pagar_3  = preço_a_pagar(15, desconto)
preço_a_pagar_4  = preço_a_pagar(38, desconto)
preço_a_pagar_5  = preço_a_pagar(92, desconto)
preço_a_pagar_6  = preço_a_pagar(30, desconto)
preço_a_pagar_7  = preço_a_pagar(86, desconto)
preço_a_pagar_8  = preço_a_pagar(68, desconto)
preço_a_pagar_9  = preço_a_pagar(6,  desconto)
preço_a_pagar_10 = preço_a_pagar(80, desconto)

print(preço_a_pagar_1)
print(preço_a_pagar_2)
print(preço_a_pagar_3)
print(preço_a_pagar_4)
print(preço_a_pagar_5)
print(preço_a_pagar_6)
print(preço_a_pagar_7)
print(preço_a_pagar_8)
print(preço_a_pagar_9)
print(preço_a_pagar_10)
```

Output

```
87.3
63.0
13.5
34.2
82.8
27.0
77.4
61.2
5.4
72.0
```

Inaceitável

- E se fossem mil produtos? O programa teria que ter pelo menos duas mil linhas (uma linha para calcular cada preço a pagar e outra para fazer `print`! E isso é inaceitável!

Solução

- Usar uma lista e um ciclo `for`.

Programa (no ficheiro `exercicio_6_muitos_precos_v2.py`)

Escreva e execute este programa. Confirme o seu *output*.

```
def preço_a_pagar(preço, desconto):

    return preço * (1.0 - desconto/100)

desconto = 10 # este valor é em percentagem

lista_de_preços = [97, 70, 15, 38, 92, 30, 86, 68, 6, 80]

for preço in lista_de_preços:
    valor_preço_a_pagar = preço_a_pagar(preço, desconto)
    print(valor_preço_a_pagar)
```

Output

```
87.3
63.0
13.5
34.2
82.8
27.0
77.4
61.2
5.4
72.0
```

Para fazer

Considere o programa Python 3 que se segue, assim como o seu *output*. Copie o programa exatamente como está. Os `imports`, bem como as instruções `seed`, `choices` e `randint`, destinam-se exclusivamente à geração de valores pseudoaleatórios, de forma repetível.

```
from random import seed
from random import choices
from random import randint

seed(5594)

g = [73, 12, 50, 49, 88, 5, 61, 30]
z = r(g)
print(g)
print(z)

g_1 = choices(range(0, 101), k=randint(10000, 10999))
g_2 = choices(range(0, 101), k=randint(11000, 11999))
g_3 = choices(range(0, 101), k=randint(12000, 12999))
g_4 = choices(range(0, 101), k=randint(13000, 13999))
g_5 = choices(range(0, 101), k=randint(14000, 14999))

print('só para verificação dos valores pseudoaleatórios gerados')
print(g_5[0])
print(g_5[1])
print(g_5[-2])
print(g_5[-1])
```

A função `r`, tem um argumento. Uma lista com números inteiros. A função `r` retorna a quantidade de números maiores ou iguais a cinquenta na lista de números inteiros.
O *output* que se segue ilustra o funcionamento descrito.

```
[73, 12, 50, 49, 88, 5, 61, 30]
4
só para verificação dos valores pseudoaleatórios gerados
43
75
89
72
```

Acrescente ao programa a função `r`, assim como o código que lhe permita indicar se as afirmações que se seguem são verdadeiras ou falsas.

1.1

A quantidade de números maiores ou iguais a cinquenta, na lista `g_1`, é `5411`.

1.2

A quantidade de números maiores ou iguais a cinquenta, na lista `g_2`, é `5831`.

1.3

A quantidade de números maiores ou iguais a cinquenta, na lista `g_5`, é `7200`.

1.4

A quantidade de números maiores ou iguais a cinquenta, na lista `g_3`, é `6328`.

1.5

A quantidade de números maiores ou iguais a cinquenta, na lista `g_4`, é `6681`.